Engenharia de Software é um conjunto de etapas que envolve:

- métodos
- ferramentas
- procedimentos

Essas etapas são conhecidas como componentes de CICLO DE VIDA DE SOFTWARE ou Processo de Software.

Ciclos de Vida de Software

Descrevem como um software deve ser desenvolvido.

Basicamente definem a ordem global das atividades envolvidas em um contexto de projeto de software e propõe uma estratégia de desenvolvimento que pode ser aplicada a um determinado contexto de projeto de software.

Para escolha de um Ciclo de Vida de Software, atentaremos para:

- natureza do projeto e da aplicação;
- métodos e ferramentas a serem usados;
- controles e produtos que precisam ser entregues.

Diferenças entre os Ciclos de Vida de Software

Enfoque dado pelo modelo

Por exemplo, no modelo Cascata o enfoque é dado na documentação e no Modelo Espiral o enfoque é dado nos

riscos.

Estratégia de desenvolvimento

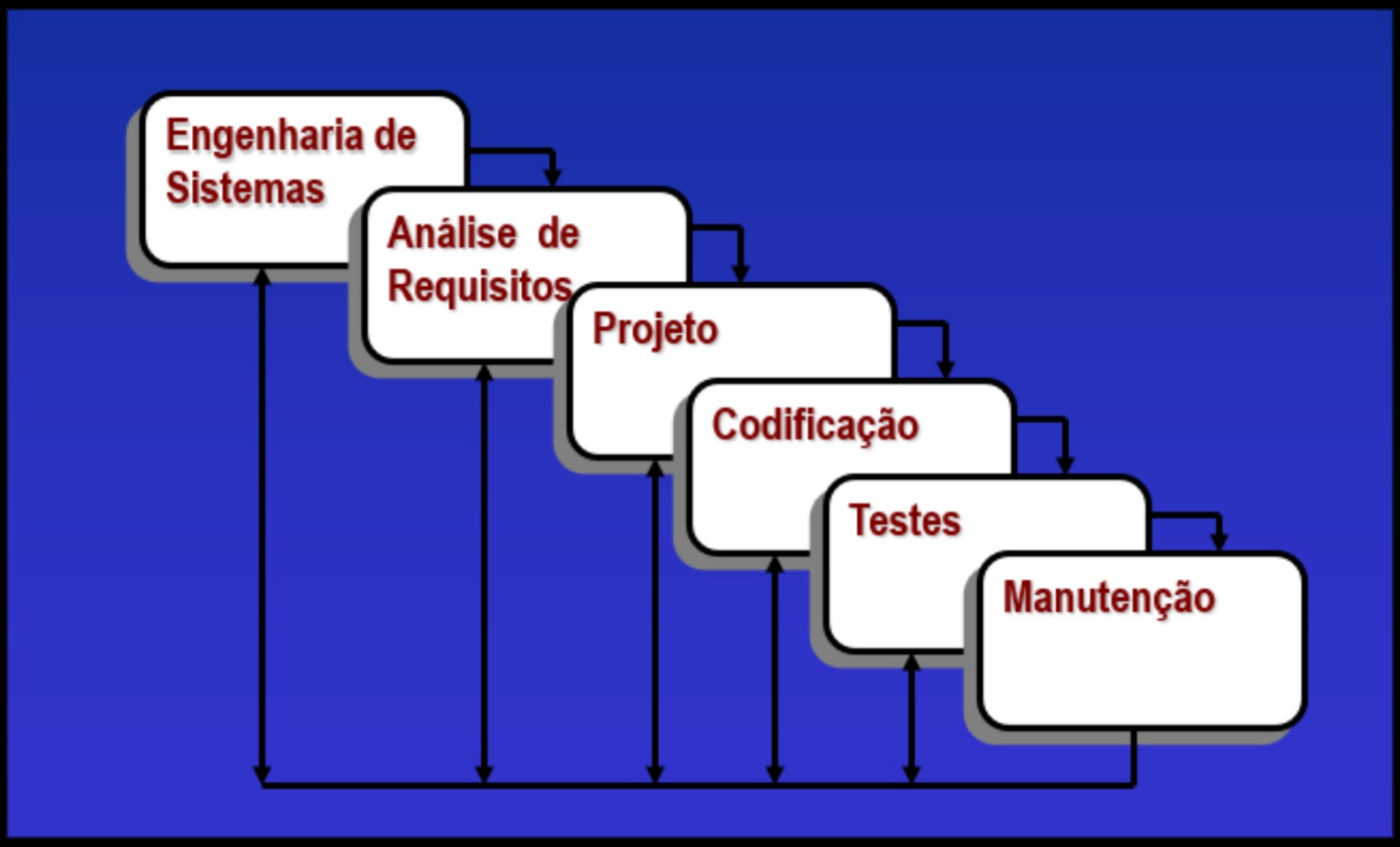
Define a disposição das atividades que deverão ser executadas para atingir um objetivo em um contexto de projeto de desenvolvimento de software.

Exemplo:

- Linear (uma atividade aós a outra) como no ciclo de vida Cascata puro ou iterativa (um conjunto de atividades é repetida várias vezes até atingir o seu objetivo) como nos modelos incrementais.
- Em Paralelo, como a prototipação, ou reunuir as características de modelos de clico de vida lineares e iterativos.

Ciclo de Vida Clássico (Cascata)

- Modelo mais antigo e o mais amplamente usado da engenharia de software;
- Modelo em função do clico da engenharia convencional;
- Requer uma abordagem sistemática, sequencial ao desenvolvimento de software;
- Consiste na execução das atividades de desenvolvimento de software em uma sequência ordenada. Desta forma, a passagem para determinada atividade exige como critério a finalização da atividade imediatamente .



Análise e Engenharia de Sistemas

- Envolve a coleta de requisitos em nível do sistema, pequena quantidade de projeto e análise de alto nível;
- Visão essencial quando o software deve fazer interface com outros elementos (hardware, pessoas e banco de dados).

Análise de Requisitos de Software

- Processo de coleta dos requisitos é intensificado e concentrado especificamento no software;
- Deve-se compreender o domínio da informação, a função, desempenho e interfaces exigidos;
- Os requisitos (para o sistema e para o software) são documentados e revistos com o cliente.

Projeto

- Tradução dos requisitos do software par um conjunto de representações que podem ser avaliadas quanto à qualidade, antes que a codificação se inicie.

Se concentra em 4 atributos do programa:

1. Estrutura de Dados;
2. Arquitetura de Software;
3. Detalhes Procedimentais;
4. Caracterização de Interfaces.

Codificação

- Tradução das representações do projeto para uma linguagem "artificial" resultando em instruções executáveis pelo computador.

Testes

Concentram-se:

- Nos aspectos lógicos internos do software, garantindo que todas as instruções tenham sido testadas;
- Nos aspectos funcionais exeternos, para descobrir erros e garantir que a entrada definida produza resultados que concordem com os esperados.

Manutenção

- O software deverá sofrer mudanças depois que for entregue ao cliente;
- Causas das mudanças: erros, adaptação do software para acomodar mudanças em seu ambiente externo e exigência do cliente para acréscimos funcionais e de desempenho.

Vantagens no Ciclo de Vida Clássico (Cascata)

- É empregado quando se domina o tipo de software que se construirá;
- Domínio nas tecnologias envolvidas (métodos, técnicas e ferramentas).

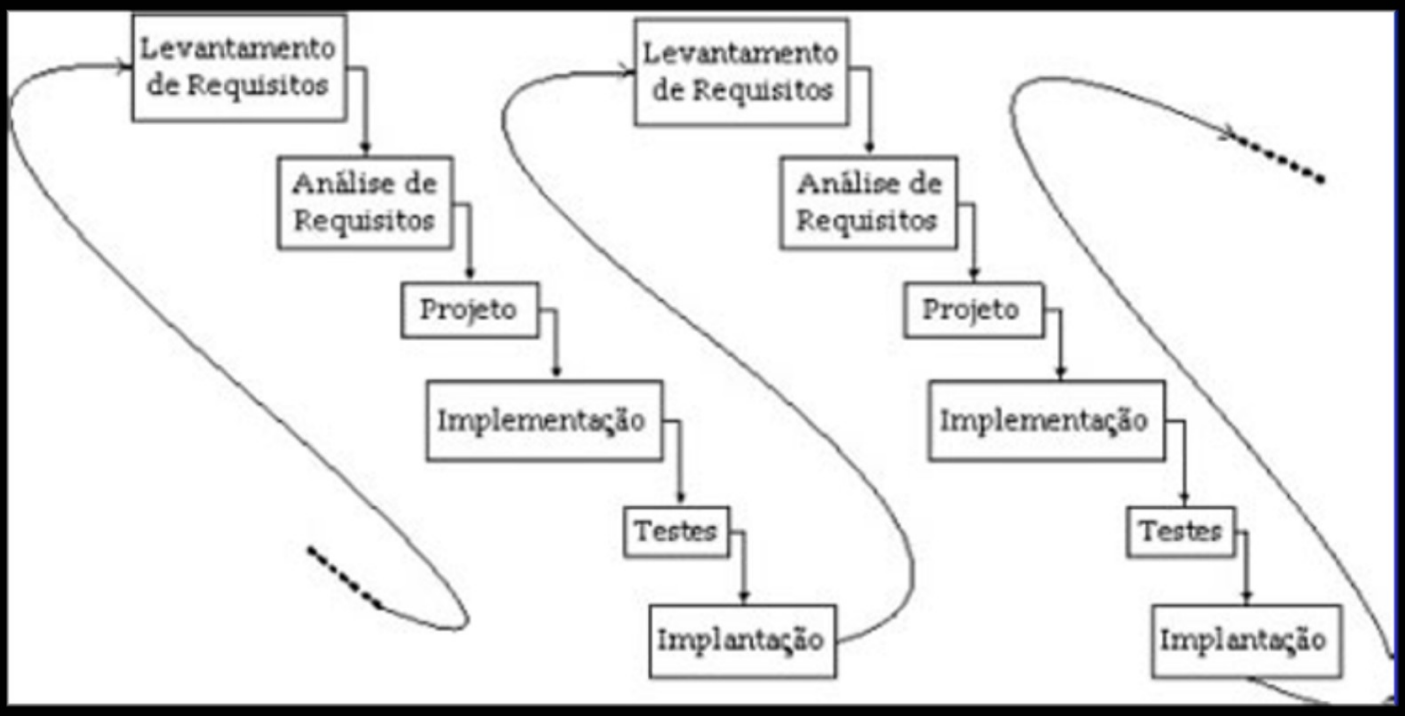
Desvantagens com o Ciclo de Vida Clássico (Cascata)

- Projetos reais raramente seguem o fluxo sequencial que o modelo propõe;
- Loog no início é difícil estabelecer explicitamente todos os requisitos;
- O cliente deve ter paciência. Uma versão executável do software só fica disponível numa etapa avançada do desenvolvimento.

Incremental

Nesse modelo o sistema é entregue por partes, módulos.

Nesse caso o cliente solicita o desenvolvimento de um grande sistema e determina a ordem de prioridades para às entregas, como por exemplo poderiámos dizer que uma empresa possui vários departamentos, entre eles o financeiro, segundo a especificação do cliente ele deseja que esse departamento seja o primeiro a operar com o sistema.



Vantagem

Subdivisão da equipe que não precisa estar totalmente presente em todas as fases realizadas, os recursos podem ser alocados em outros projetos e só convocados para este projeto quando necessário.

Diferencial entre Cascata e Incremental

- Implementação: Etapa onde se instala o .exe no ambiente computacional do cliente, prepara-se uma

base de dados para testes (aplicação de treinamentos), depois uma base de produção (receber os CRUDs da operação do SW);

- Entrega do SW em estágios por múdulos. Exemplo: Sistemas de Gestão Empresacial, ERP.

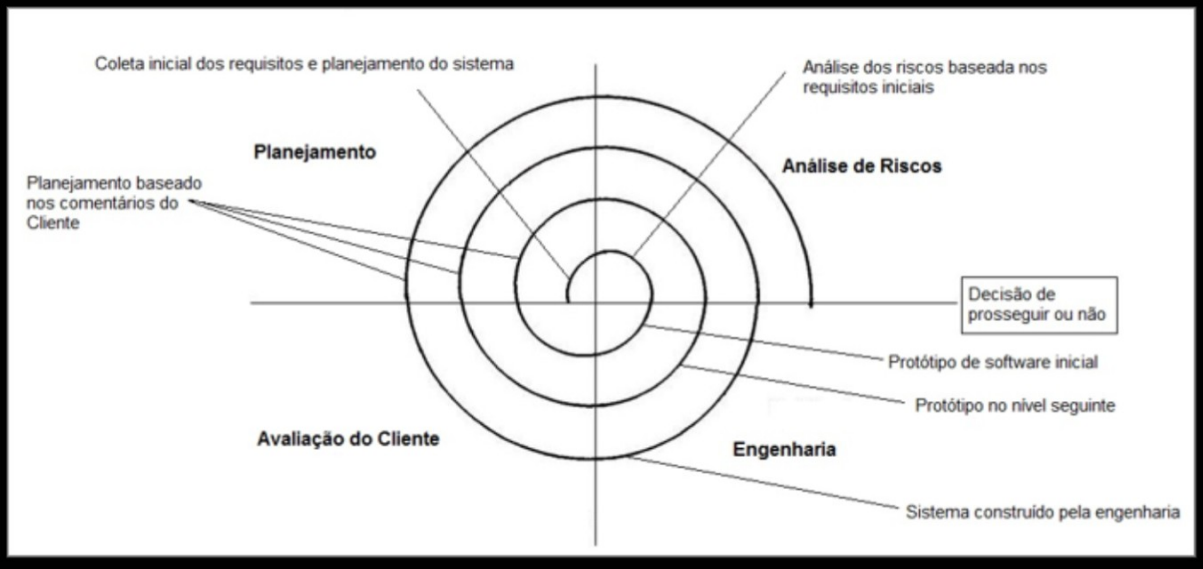
Ciclo de Vida em Espiral

Engloba as melhores características do ciclo de vida Clássico e da Prototipação, adicionando um novo elemento: a Análise de Risco.

Segue a abordagem de passos sistemáticos do Ciclo de Vida Clássico incorporando-os numa estrutura iterativa que reflete mais realisticamente o mundo real.

Usa a Prototipação, em qualquer etapa da evolução do projeto, como mecanismo de redução de riscos.

Obs: Prototipação é um processo que possibilita que o desenvolver crie um modelo do software que deve ser construído.



Atividades do Ciclo de Vida em Espiral

- Comunicação com o Cliente: define as tarefas necessárias para manter a comunicação entre o desenvolvedor e o cliente;
- Planejamento: define recursos, prazos, objetivos, alternativas e restrições;
- Análise de Risco: análise das alternativas e identificação / resolução dos riscos técnicos e operacionais;
- Construção: define tarefas necessárias para construir, testar, instalar e fornecer apoio ao usuário;
- Avaliação do Cliente: avaliação das representações do software criadas durante o estágio de engenharia e implementadas durante o estágio de instalação.

Vantagens no Ciclo de Vida em Espiral

- Quanto mais tempos e recursos forem destinados, ou seja, quanto mais iterações na espiral, menor serão os riscos sobre o projeto;
- Execução de atividades de verificação representes ao final de cada iteração que permitem um melhor controle gerencial sobre o projeto.